



WEEK 3 MODULE

SORTING PROGRAMMING ASSIGNMENT



Name : Annpurna Sahoo

Roll Number : 2404921530048

Branch : B.Tech CSE (AIML)

Year : 3rd Year

Section : AIML

Subject : Sorting Programming

Submission Date : 6.07.26



Topic 1: -

Module 3

Ques 1 (Easy): Merge Sort Algorithm

↓
use divide &
conquer

Algorithm: -

- 1) Start the program
- 2) Read the number of package values (N)
- 3) Store all package values in an array
- 4) Divide the array into two smaller halves recursively.
- 5) Sort each half using merge sort
- 6) Merge the two sorted halves into one sorted array.
- 7) Repeat until the entire array is sorted.
- 8) Print the sorted package values in ascending order.
- 9) End the program

C++ Code

```
#include <iostream>
using namespace std;

void merge (int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
```

```
for (int i = 0; i < n1; i++)
    L[i] = arr[left + i];
```

```
for (int j = 0; j < n2; j++)
    R[j] = arr[mid + 1 + j];
```

```
int i = 0, j = 0, k = left;
```

```
while (i < n1 & j < n2)
```

```
{
    if (L[i] < R[j])
```

```
        arr[k++] = L[i++];
```

```
    else arr[k++] = R[j++];
```

```
while (i < n1)
```

```
    arr[k++] = L[i++];
```

```
while (j < n2)
```

```
    arr[k++] = R[j++];
```

```
y
void mergeSort (int arr[], int left, int right)
```

```
{
    if (left < right)
```

```
{
    int mid = (left + (right - left) / 2);
```

```
    mergeSort (arr, left, mid);
```

```
    mergeSort (arr, mid + 1, right);
```

```
    mergeSort (arr, left, mid, right);
```

y

y

```

int n;
cin >> n;
int arr[n];
for (int i = 0; i < n; i++)
    cin >> arr[i];
mergeSort(arr, 0, n-1);
for (int i = 0; i < n; i++)
    cout << arr[i] << " ";
return 0;

```

y

4) Sample Input -

6
450 1200 800 650 300 1500

5) Sample Output -

300 450 650 800 1200 1500

6) Time Complexity -

Best Case - $O(n \log n)$

Avg. Case - $O(n \log n)$

Worst Case - $O(n \log n)$

7) Space Complexity - $O(n)$

```
1 #include <iostream>
2 using namespace std;
3 void merge(int arr[],int left,int mid,int right){
4     int n1=mid-left+1;
5     int n2=right-mid;
6     int L[n1],R[n2];
7     for(int i=0;i<n1;i++)
8         L[i]=arr[left+i];
9
10    for(int j=0;j<n2;j++)
11        R[j]=arr[mid+1+j];
12
13    int i=0,j=0,k=left;
14    while(i<n1 && j<n2){
15        if(L[i]<=R[j])
16            arr[k++]=L[i++];
17        else
18            arr[k++]=R[j++];
19    }
20    while(i<n1)
21        arr[k++]=L[i++];
22    while(j<n2)
23        arr[k++]=R[j++];
24 }
25
26 void mergeSort(int arr[],int left,int right){
27     if(left<right)
28         f
```

main.cpp

```
20 while(i<n1)
21     arr[k++]=L[i++];
22 while(j<n2)
23     arr[k++]=R[j++];
24 }
25
26 void mergeSort(int arr[],int left,int right){
27     if(left<right)
28     {
29         int mid=left+(right-left)/2;
30         mergeSort(arr,left,mid);
31         mergeSort(arr,mid+1,right);
32         merge(arr,left,mid,right);
33     }
34 }
35
36 int main() {
37     int n;
38     cin >> n;
39     int arr[n];
40     for(int i=0;i<n;i++)
41         cin >> arr[i];
42     mergeSort(arr,0,n-1);
43     for(int i=0;i<n;i++)
44         cout << arr[i]<<" ";
45     return 0;
46 }
47
```

```
6
450 1200 800 650 300 1500
300 450 650 800 1200 1500
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Ques 2. (Medium)

Logic -

~~1) Read the no. of processing times (N)~~

- 1) Start the program.
- 2) Read the value of N.
- 3) Input N processing times into an array.
- 4) Apply merge sort to sort the array in ascending order.
- 5) Display the sorted array.
- 6) Find the median of the sorted array.
- 7) ~~Count~~ Count the elements greater than the median.
- 8) Calculate the difference btw the max. & min. elements.
- 9) Display the median, count of elements greater than median, and the difference.
- 10) End of program.

N=7

Array

12 25 18 30 15 22 40

↓

12 25 18

↓

12 25 18

↓

25 18

30 15 22 40

↓

30 15 22 40

↓

30 15

↓

22 40

$$SS = 51 - 0.11 = 50.89$$

3) Merge (add & sort them)

Left side

25 18

↓

18 25

↓

12 18 25

Right side

30 15

↓

15 30

↓

15 22 30 40

Final Sorted Array

12 15 18 22 25 30 40

0 1 2 3 4 5 6 → indexing

4) Median = $arr[N/2]$

$N=7$ (Odd)

Median = 22

Greater than 22

25

30

40

]- total = 3

difference →

Min = 12

Max = 40

diff. = 40 - 12 = 28

C++ Code :-

```
#include <iostream>
using namespace std;
```

```
void merge (int arr[], int l, int mid, int r) {
```

```
    int n1 = mid - l + 1;
```

```
    int n2 = r - mid;
```

```

int L[n1], R[n2];
for (int i = 0; i < n1; i++)
    L[i] = arr[i + 1];
for (int j = 0; j < n2; j++)
    R[j] = arr[mid + 1 + j];

```

```

int l = 0, j = 0, k = 1;
while (i < n1 && j < n2) {
    if (L[i] <= R[j])

```

```

        arr[k++] = L[i++];

```

else

```

        arr[k++] = R[j++];

```

```

}

```

```

}

```

```

while (i < n1)

```

```

    arr[k++] = L[i++];

```

```

}

```

```

}

```

```

while (j < n2)

```

```

    arr[k++] = R[j++];

```

```

}

```

```

}

```

void mergeSort (int arr[], int left, int right) {

```

    if (left < right) {
        mergeSort (arr, left, mid);
        mergeSort (arr, mid + 1, right);
        merge (arr, left, mid, right);
    }
}

```

```
int mid = (l + r) / 2;
```

```
mergeSort(arr, l, mid);
```

```
mergeSort(arr, mid + 1, r);
```

```
merge(arr, l, mid, r);
```

```
}
```

```
y
```

```
int main()
```

```
{
```

```
int N;
```

```
cin >> N;
```

```
vector<int> arr(N);
```

```
for (int i = 0; i < N; i++)
```

```
cin >> arr[i];
```

```
mergeSort(arr, 0, N - 1);
```

```
for (int i = 0; i < N; i++)
```

```
cout << arr[i] << " "; cout
```

```
cout << endl; cin >> arr[i] << " ";
```

```
cout << endl;
```

```
// Median
```

```
int median = arr[N/2]; arr[N/2];
```

```
if (N % 2 == 1)
```

```
median = arr[N/2];
```

```
else median = (arr[N/2] + arr[N/2 + 1]) / 2;
```

```
cout << median << endl;
```

```
int count = 0;
```

```
for (int i = 0; i < N; i++) for (int i = 0; i < N; i++)
```

```
if (arr[i] > median)
```

```
count++;
```

```

// Difference
cout << " Difference: " << arr[N-1] - arr[0] << endl;

return 0;

```

time complexity - $O(n \log n)$
 space complexity - $O(n)$

* Sample Input -

12 25 18 30 15 22 40

* Sample output

12 15 18 22 25 30 40

Median: 22

Greater than median: 3

Difference: 28

(arr[6] - arr[0])

;

(arr[6] - arr[0])

(arr[6] - arr[0])

(arr[6] - arr[0])

(arr[6] - arr[0])

(arr[6] - arr[0])



main.cpp

```
1 #include<iostream>
2 using namespace std;
3 void merge(int arr[], int l, int mid, int r){
4     int n1=mid-l+1;
5     int n2=r-mid;
6     int L[n1],R[n2];
7     for(int i=0;i<n1;i++)
8         L[i]=arr[l+i];
9
10    for(int j=0;j<n2;j++)
11        R[j]=arr[mid+1+j];
12
13    int i=0,j=0,k=l;
14    while(i<n1 && j<n2){
15        if(L[i]<=R[j]){
16            arr[k++]=L[i++];
17        }
18        else
19        {
20            arr[k++]=R[j++];
21        }
22    }
23    while(i<n1)
24        arr[k++]=L[i++];
25    while(j<n2)
26        arr[k++]=R[j++];
27 }
```



main.cpp

PDF

Image to PDF

```
31
32 void mergeSort(int arr[],int l,int r){
33     if(l<r)
34     {
35         int mid=(l+r)/2;
36         mergeSort(arr,l,mid);
37         mergeSort(arr,mid+1,r);
38         merge(arr,l,mid,r);
39     }
40 }
41
42 int main() {
43     int N;
44     cin >> N;
45     int arr[N];
46     for(int i=0;i<N;i++)
47         cin >> arr[i];
48
49     mergeSort(arr,0,N-1);
50
51     for(int i=0;i<N;i++)
52         cout << arr[i]<<" ";
53     cout << endl;
54
55     int median=arr[N/2];
56
```



main.cpp

```
42 int main() {
43     int N;
44     cin >> N;
45     int arr[N];
46     for(int i=0;i<N;i++)
47         cin >> arr[i];
48
49     mergeSort(arr,0,N-1);
50
51     for(int i=0;i<N;i++)
52         cout << arr[i]<<" ";
53     cout << endl;
54
55     int median=arr[N/2];
56
57     cout << "Median:"<< median << endl;
58     int count = 0;
59     for(int i=0;i<N;i++)
60     {
61         if(arr[i]>median)
62             count++;
63     }
64     cout << "Greater than median:" << count << endl;
65     //Difference
66     cout << "Difference:"<< arr[N-1]-arr[0]<<endl;
67     return 0;
68
69 }
```

input

```
12 15 18 22 25 30 40
Median:22
Greater than median:3
Difference:28
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

close ad [x]

AdChoices

MIC Your Trading Edge

Trade the Oil Momentum

- Leverage 1:500
- Spread from O.D pipe

Trade Now

Trading derivatives involves high risk to your capital.



Topic 2: Quick Sort

Quest 1 (Easy)

Algorithm -

- 1) Read the value of N .
- 2) Read the response times.
- 3) Select the last element as the pivot.
- 4) Partition the array around the pivot.
- 5) Recursively apply Quick Sort to the left & right subarray.
- 6) Print the sorted response times.

C++ Code

```
#include <iostream>
using namespace std;

int partition (int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i+1;
}

void quickSort (int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi-1);
        quickSort(arr, pi+1, high);
    }
}
```


main.cpp

```

1  #include<iostream>
2  using namespace std;
3  int partition(int arr[],int low,int high)
4  {
5      int pivot=arr[high];
6      int i=low-1;
7      for(int j=low;j<high;j++)
8      {
9          if(arr[j]<pivot)
10         {
11             i++;
12             swap(arr[i],arr[j]);
13         }
14     }
15     swap(arr[i+1],arr[high]);
16     return i+1;
17 }
18 void quickSort(int arr[],int low,int high){
19     if(low < high) {
20         int pi = partition(arr,low,high);
21         quickSort(arr, low, pi-1);
22         quickSort(arr, pi+1 , high);
23     }
24 }
25 }
26 int main()
27 {
28     int N;

```

```

6
120 45 78 200 60 95
95 45 120 200 78 60

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

input

main.cpp

```
11     swap(arr[i], arr[j]);
12 }
13 swap(arr[i+1], arr[high]);
14 return i+1;
15 }
16 void quickSort(int arr[], int low, int high){
17     if(low < high) {
18         int pi = partition(arr, low, high);
19         quickSort(arr, low, pi-1);
20         quickSort(arr, pi+1, high);
21     }
22 }
23 int main()
24 {
25     int N;
26     cin >> N;
27     int arr[N];
28     for(int i=0; i<N; i++)
29         cin >> arr[i];
30
31     quickSort(arr, 0, N-1);
32     for(int i=0; i<N; i++)
33         cout << arr[i] << " ";
34     return 0;
35 }
```

close ad [x]

Input

```
6
120 45 78 200 60 95
95 45 120 200 78 60
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Algorithm

- 1) Read the value of N
- 2) Read all trade values
- 3) Sort the array in descending order using Quick Sort.
- 4) Display the sorted trade values.
- 5) print the top 5 highest trade values.
- 6) Calculate the average of the top 5 values.
- 7) Calculate the overall average of all trade values.
- 8) Count the values greater than the overall average.
- 9) Print all required results.

C++ Code -

```
#include <iostream>
using namespace std;

int partition (int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] > pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
}
```

```

swap(arr[i+1], arr[high]);
return i+1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, pi+1, high);
        quickSort(arr, low, pi-1);
    }
}

int main() {
    int n;
    cin >> n;
    int arr[n];
    int sum = 0;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        sum += arr[i];
    }

    quickSort(arr, 0, n-1);

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    cout << endl;
    cout << "sum of top values";
    // top 5 print top values;
    int topSum = 0;
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    cout << endl;
    int topSum = 0;
    for (int i = 0; i < n; i++) {
        topSum += arr[i];
    }
    cout << topSum << endl;
}

```

```

cout << arr[i] << " ";
topSum += arr[i];
y
cout << endl;
// Average of top 5
double avg top 5 = topSum / 5;
cout << "Average of top 5 : " << topSum / 5 << endl;

```

// ~~Overall~~ Average

double ~~overall~~ Avg = (double) Sum / N;

// Count values above overall average

int count = 0;

for (int i = 0; i < N; i++)

{ if (arr[i] > ~~overall~~ Avg)

count++;

}

cout << "Values Above Overall Average:"

<< count << endl;

return 0;

y

Sample Input

8

500

1200

700

2000

900

1500

3000

2000

Sample output

3000

2500

2000

1500

1200

900

700

500

top 5: 3000

2500

2000

1500

1200

900

700

500

Average of top 5: 2048 2040
Values Above Average: 434

Keep the balance of 11.

Build a new top 5 from the new.

Remove the new 5 from the top 5.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.

Repeat the process.



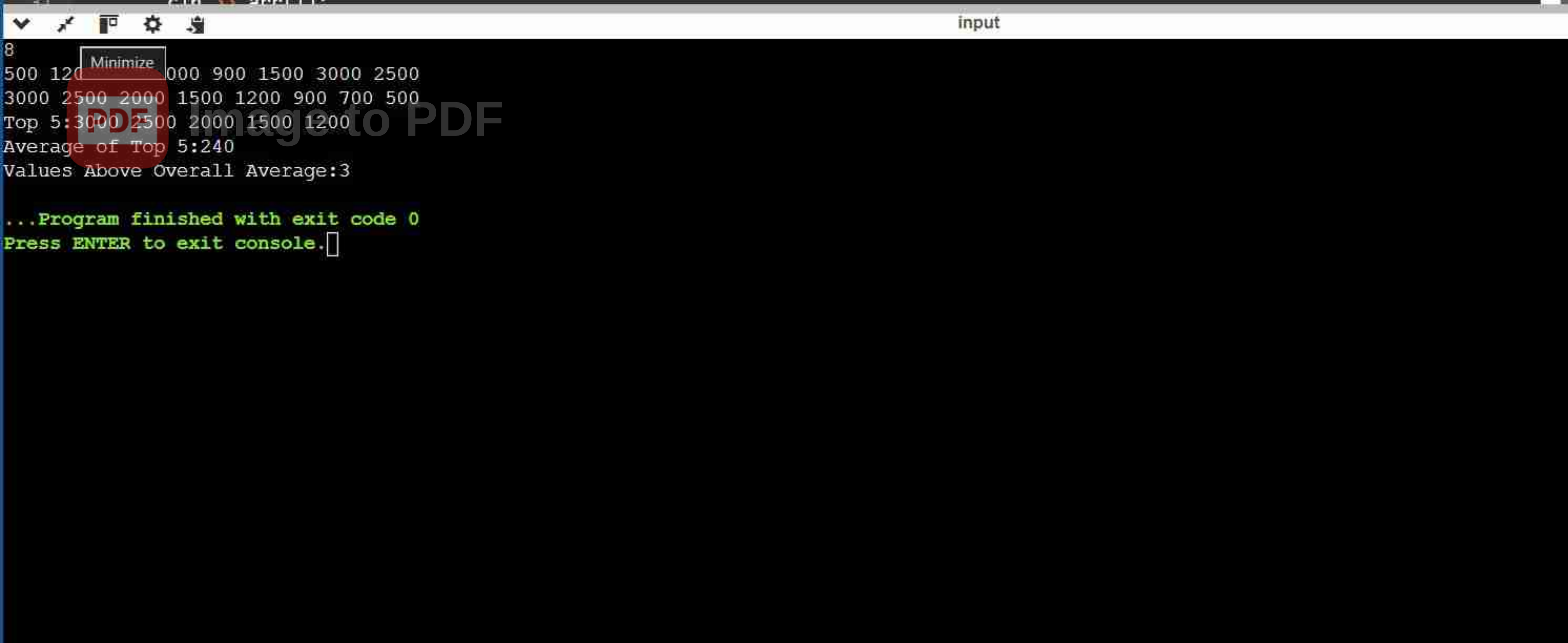
main.cpp

```
1 #include<iostream>
2 using namespace std;
3 int partition(int arr[],int low,int high){
4     int pivot = arr[high];
5     int i = low - 1;
6     for(int j=low;j<high;j++){
7         if(arr[j]>pivot)
8         {
9             i++;
10            swap(arr[i],arr[j]);
11        }
12    }
13    swap(arr[i+1],arr[high]);
14    return i+1;
15 }
16 void quickSort(int arr[],int low,int high){
17     if(low < high)
18     {
19         int pi = partition(arr,low,high);
20         quickSort(arr, low, pi - 1);
21         quickSort(arr, pi + 1, high);
22     }
23 }
24 int main() {
25     int N;
26     cin >> N;
27     int arr[N];
28     int sum=0;
29     for(int i=0;i<N;i++)
30     {
31         cin >> arr[i];
32         sum+=arr[i];
33     }
34     quickSort(arr,0,N-1);
35     for(int i=0;i<N;i++)
36         cout<< arr[i]<< " ";
```



main.cpp

```
26 cin >> N;
27 int arr[N];
28 int sum=0;
29 for(int i=0;i<N;i++)
30 {
31     cin >> arr[i];
32     sum+=arr[i];
33 }
34 quickSort(arr,0,N-1);
35 for(int i=0;i<N;i++)
36     cout<< arr[i] << " ";
37 cout << endl;
38
39
40 int top5sum = 0;
41 cout << "Top 5:";
42 for(int i=0;i<5;i++)
43 {
44     cout<< arr[i] << " ";
45     top5sum+=arr[i];
46 }
47 cout << endl;
48
49 cout << "Average of Top 5:"<< top5sum /5 << endl;
50
51 double avg=(double)sum/N;
52 int count=0;
53 for(int i=0;i<N;i++){
54     if(arr[i]>avg)
55         count++;
56 }
57
58 cout <<"Values Above Overall Average:" << count;
59 return 0;
60
61 }
```



Topic 3 : Heap Sort

Ques 1. (Easy)

Algorithm

- 1) Read the value of N .
- 2) Read all player scores.
- 3) Build a max-heap from the array.
- 4) Swap the root with the last element.
- 5) Reduce the heap size by one.
- 6) Heapify the remaining heap.
- 7) Repeat until all elements are sorted.
- 8) Print the sorted scores.

C++ Code

```
#include <iostream>
using namespace std;
void heapify (int arr[], int n, int i) {
    int largest = i;
    int left = 2*i+1;
    int right = 2*i+2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (left < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
```

```

void heapSort (int arr[], int n) {
    // Build max-heap
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n - 1; i > 0; i--) {
        swap(arr[i], arr[0]);
        heapify(arr, i, 0);
    }
}

int main () {
    int N;
    cin >> N;

    int arr[N];
    for (int i = 0; i < N; i++)
        cin >> arr[i];

    heapSort (arr, N);

    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";

    return 0;
}

```

Sample Input

6

850 1200

Sample Output

600 650 950 1000 1200 1500



Run

Debug

Stop

Share

Save

{ } Beautify



Language C++

main.cpp

```

1 #include <iostream>
2 using namespace std;
3
4 void heapify(int arr[], int n, int i) {
5     int largest = i;
6     int left = 2 * i + 1;
7     int right = 2 * i + 2;
8
9     if (left < n && arr[left] > arr[largest])
10         largest = left;
11
12     if (right < n && arr[right] > arr[largest])
13         largest = right;
14
15     if (largest != i) {
16         swap(arr[i], arr[largest]);
17         heapify(arr, n, largest);
18     }
19 }
20
21 void heapSort(int arr[], int n) {
22     for (int i = n / 2 - 1; i >= 0; i--)
23         heapify(arr, n, i);
24
25     for (int i = n - 1; i > 0; i--) {
26         swap(arr[0], arr[i]);
27         heapify(arr, i, 0);
28     }
29 }
30
31 int main() {
32     int N;

```

32 int N;
33 cin >> N;
34 int arr[N];
35
36 for (int i = 0; i < N; i++)
37 cin >> arr[i];
38
39 heapSort(arr, N);
40
41 for (int i = 0; i < N; i++)
42 cout << arr[i] << " ";
43
44 return 0;
45 }



```
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500

...Program finished with exit code 0
Press ENTER to exit console.
```

Que 2 - (Medium)

Algorithm

- 1) Read the number of response times (N)
- 2) Read all response times into an array.
- 3) Build a max-heap sort to sort the array in ascending order.
- 5) Display the sorted response times.
- 6) Find the fastest response time (first element)
- 7) Find the slowest response time (last element)
- 8) Calculate the average response times (least than average)
- 9) Count the number of response times faster than average (percentage of cases faster than average)
- 10) Calculate the percentage of cases faster than average.
- 11) Display all results.

C++ Code

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;
void heapify (vector<int> &v, int n, int i)
```

```
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
```

```

if (left < n && arr[left] > arr[largest])
    largest = left;
if (right < n && arr[right] > arr[largest])
    largest = right;
if (largest != i) {
    swap(arr[i], arr[largest]);
    heapify(arr, n, largest);
}
}

```

```

void heapSort (Vector<int> &arr) {
    int n = arr.size();
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

```

```

int main() {
    int n;
    cin >> n;
    Vector<int> arr(n); int arr[n];
    int sum = 0;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        sum += arr[i];
    }
}

```

```

}
heapSort(arr, n);
// Sorted response times

```

```

for (int i = 0; i < N; i++)
{
    cout << arr[i] << " ";
    cout << endl;
    cout << "fastest: " << arr[0] << endl;
    cout << "slowest: " << arr[N-1] << endl;
    double average = sum / N;
}

```

```

int count = 0;
for (int i = 0; i < N; i++)
{
    if (arr[i] < average)
        count++;
}

```

```

double percentage = (count * 100.0) / N;
cout << "fastest: " << arr[0] << endl;
cout << "slowest: " << arr[N-1] << endl;
cout << "Average: " << average << endl;
cout << "Cases faster than Average: " << count << endl;

```

```

cout << "fixed << setprecision(2);
cout << "percentage: " << percentage << "%";
cout << "percentage: " << percentage << "%";
cout << endl;
return 0;
}

```

y

Input Sample -

8	12	15	9	20	5	10	8
---	----	----	---	----	---	----	---

Output Sample

5 7 8 9 10 12 15 20

Fastest : 5

Slowest : 20

Average : 10.75

Cases faster than Average : 5

percentage : 62.50%

main.cpp

```

1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 void heapify(int arr[], int n, int i) {
6     int largest = i;
7     int left = 2 * i + 1;
8     int right = 2 * i + 2;
9
10    if (left < n && arr[left] > arr[largest])
11        largest = left;
12
13    if (right < n && arr[right] > arr[largest])
14        largest = right;
15
16    if (largest != i) {
17        swap(arr[i], arr[largest]);
18        heapify(arr, n, largest);
19    }
20 }
21
22 void heapSort(int arr[], int n) {
23     for (int i = n / 2 - 1; i >= 0; i--)
24         heapify(arr, n, i);
25
26     for (int i = n - 1; i > 0; i--) {
27         swap(arr[0], arr[i]);
28         heapify(arr, i, 0);
29     }
30 }
31
32 int main() {

```

```
33 int N;
34 cin >> N;
35 int arr[N];
36 int sum = 0;
37
38 for (int i = 0; i < N; i++) {
39     cin >> arr[i];
40     sum += arr[i];
41 }
42
43 heapSort(arr, N);
44
45 for (int i = 0; i < N; i++)
46     cout << arr[i] << " ";
47 cout << endl;
48
49 cout << "Fastest: " << arr[0] << endl;
50 cout << "Slowest: " << arr[N - 1] << endl;
51
52 double avg = (double)sum / N;
53 cout << fixed << setprecision(2);
54 cout << "Average: " << avg << endl;
55
56 int count = 0;
57 for (int i = 0; i < N; i++) {
58     if (arr[i] < avg)
59         count++;
60 }
61
62 cout << "Cases Faster Than Average: " << count << endl;
63 cout << "Percentage: " << (count * 100.0 / N) << "%";
```

```
62 cout << "Cases Faster Than Average." << endl;
63 cout << "Percentage: " << (count * 100.0 / N) << "%";
64
65 return 0;
66 }
```



PDF

Image to PDF

input

```
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

...Program finished with exit code 0
Press ENTER to exit console. 
```